YOLO Object Detection on MS COCO

Muhammad Saad Bin Zahid (24K-8046)

Introduction

This report covers the implementation of YOLO, a real-time object detection model, trained on a subset of the MS COCO 2017 dataset. The goal is to train, evaluate, and compare detection models across different configurations.

Based on Roll No. 24K-8046, the number of classes N is calculated as:

$$\mathbf{N = (46 \bmod 10) + 5 = 6 + 5 = 11 \text{ classes}}$$

The 11 selected classes are: person, car, motorcycle, bus, truck, dog, cat, bicycle, bottle, chair, and tv. These were chosen to cover a diverse mix of vehicles, animals, and everyday objects, all of which appear frequently in the COCO dataset.

Repository

The complete notebook, trained weights, comparison plots, and full qualitative detection outputs for this assignment are available at:

https://github.com/saadi1997/Deep-Leaning-YOLO-Assignment

Task 1: Architecture Analysis

Anchor Boxes

Anchor boxes are predefined shapes used by object detectors to predict bounding boxes more efficiently. Instead of predicting a box from scratch, the model adjusts these anchors to fit detected objects. Earlier YOLO versions used anchor boxes precomputed via k-means clustering on training data.

YOLOv8 uses an anchor-free approach, meaning it directly predicts the distances from a center point to the four sides of a bounding box. To improve localization accuracy, it uses Distribution Focal Loss (DFL), which treats bounding box boundaries as a probability distribution rather than fixed points. This makes detections more precise, especially for objects at varying scales.

Grid Cells

YOLO divides the input image into an S x S grid. Each grid cell is responsible for detecting objects whose center falls within it. Rather than scanning the image multiple times like earlier detectors, YOLO processes the whole image in a single forward pass, making it very fast.

In YOLOv8 with imgsz=640, the backbone generates feature maps at three different scales: 80×80, 40×40, and 20×20. Smaller grid cells are better for small objects and fine details, while larger grid cells are better for large objects. In our experiments, we used imgsz=320 for the baseline and imgsz=512 for the experiment, which changed the feature map sizes proportionally.

Non-Max Suppression (NMS)

In practice, NMS is applied after the model produces multiple candidate boxes. It keeps the highest-confidence box and removes nearby duplicate boxes based on overlap, so the final detections are cleaner and easier to interpret. For the qualitative examples in this report, predictions were displayed at a confidence threshold of 0.25.

Task 2: Model Training

Dataset Preparation

The MS COCO 2017 training annotations (instances_train2017.json) were used to filter images. For each of the 11 selected classes, up to 300 images were sampled at random (seed=42) to reduce the dominance of very frequent classes and keep the dataset manageable. This resulted in 3,230 unique training images and 22,132 annotations saved as a filtered JSON subset.

The COCO 2017 validation set (val2017) was used as the validation split and filtered to the same 11 classes, giving 3,532 validation images with 17,565 annotations. Training images were downloaded from COCO, while the validation images were taken from the existing val2017 split and organized into the YOLO folder structure. Bounding box annotations were converted from COCO format [x, y, w, h] to YOLO format [class_id, cx, cy, w, h], normalized between 0 and 1.

Model Configuration

Training was performed using the pretrained YOLOv8n (nano) model as the base. The baseline configuration was:

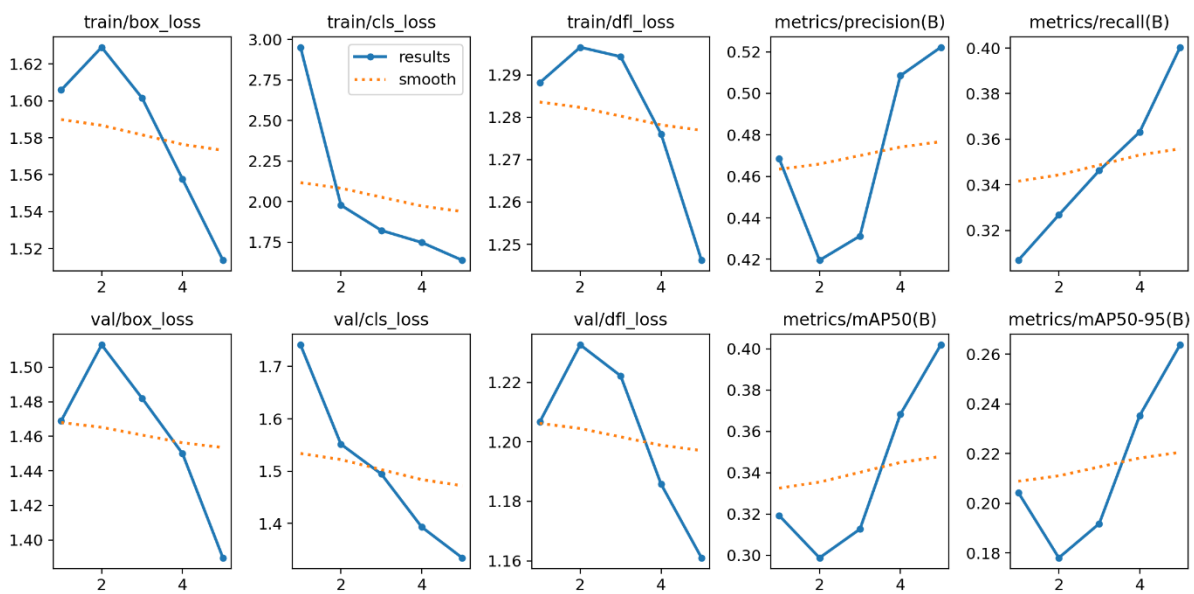
Parameter	Value
Model	YOLOv8n (pretrained)
Epochs	5
Image Size	320 x 320
Batch Size	16
Optimizer	AdamW (auto)
Device	Tesla T4 GPU
Classes	11
Training Images	3,230
Validation Images	3,532

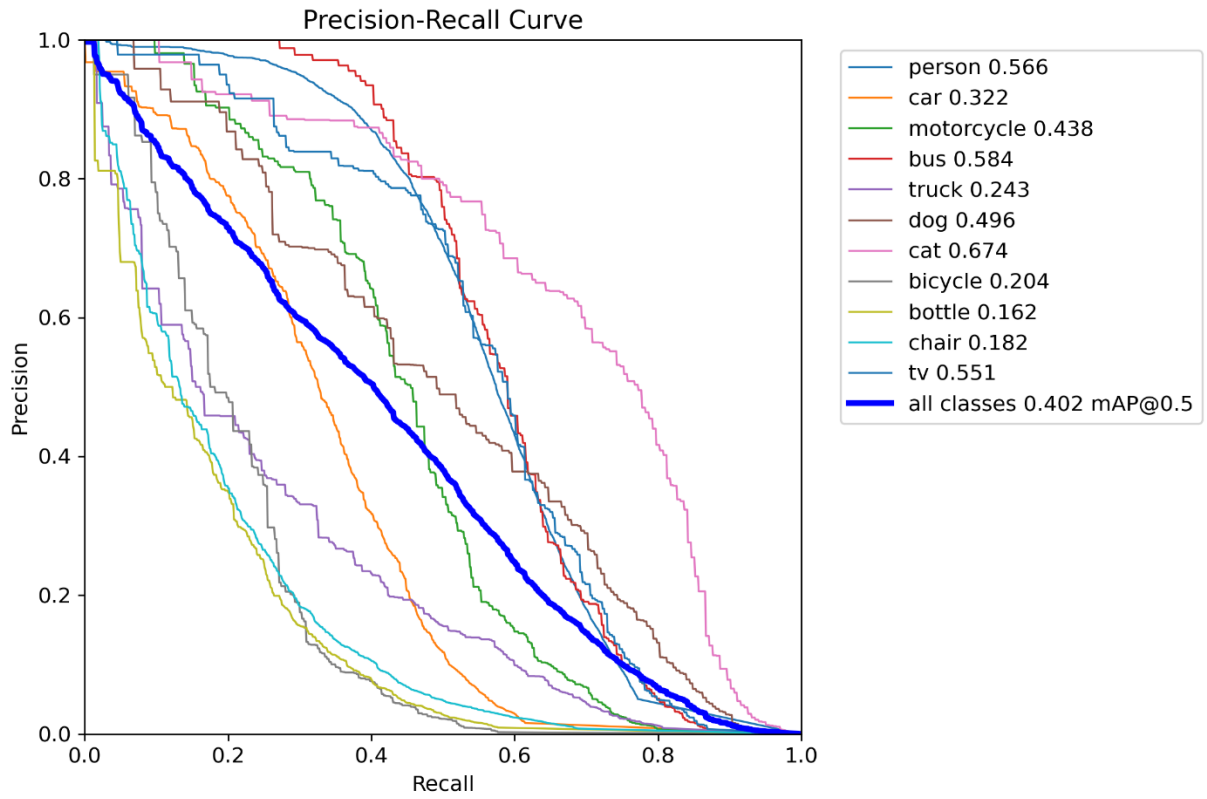
Training Results

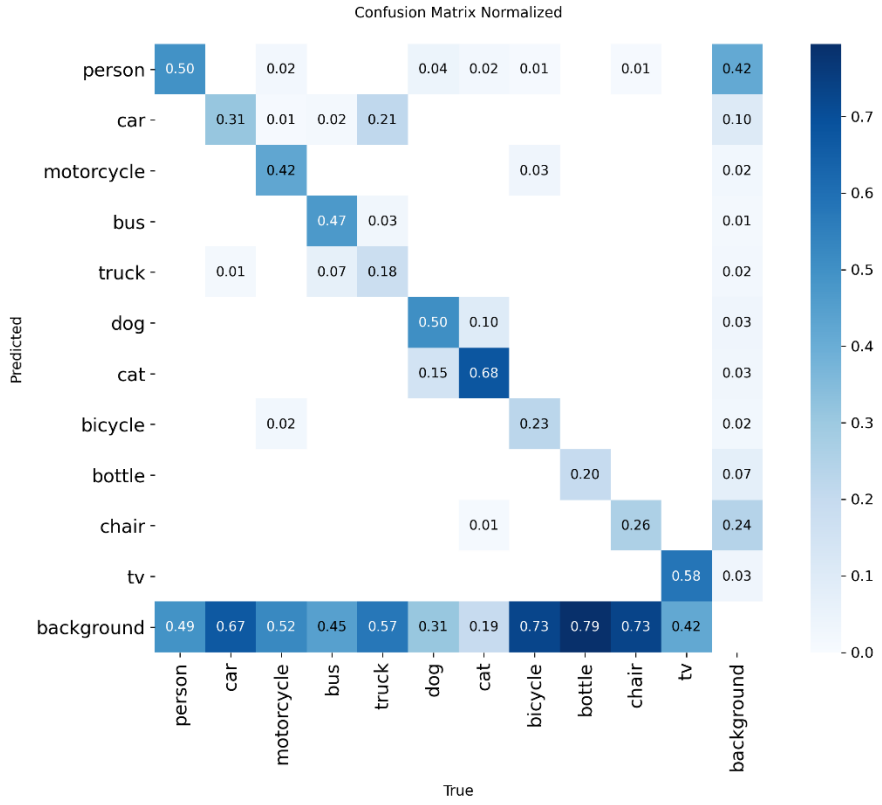
Training was completed in approximately 6 minutes on a Tesla T4 GPU. The table below summarizes per-epoch metrics:

Epoch	Box Loss	Cls Loss	DFL Loss	mAP@0.5	mAP@0.5:0.95	Precision	Recall
1/5	1.606	2.951	1.288	0.319	0.204	0.469	0.307
2/5	1.629	1.977	1.297	0.299	0.178	0.420	0.327
3/5	1.602	1.822	1.294	0.313	0.192	0.431	0.346
4/5	1.558	1.747	1.276	0.368	0.235	0.509	0.363
5/5	1.514	1.638	1.246	0.402	0.264	0.522	0.400

The classification loss (cls_loss) declined steadily from 2.951 at epoch 1 to 1.638 at epoch 5, showing the model progressively learning to distinguish between the 11 object classes. The mAP@0.5 improved from 0.319 to 0.402 across 5 epochs. The dip at epoch 2 is typical behavior in short training runs where the model transitions from general feature learning to class-specific optimization. By epoch 5, a balanced state was reached.




Precision-Recall Curve
Precision
Recall
0.0
0.2
0.4
0.6
0.8
1.0
person 0.566
car 0.322
motorcycle 0.438
bus 0.584
truck 0.243
dog 0.496
cat 0.674
bicycle 0.204
bottle 0.162
chair 0.182
tv 0.551
all classes 0.402 mAP@0.5


Confusion Matrix Normalized
Predicted
True
person 0.50 0.02 0.04 0.02 0.01 0.01 0.42
car 0.31 0.01 0.02 0.21 0.10
motorcycle 0.42 0.03 0.02
bus 0.47 0.03 0.01
truck 0.01 0.07 0.18 0.02
dog 0.50 0.10 0.03
cat 0.15 0.68 0.03
bicycle 0.02 0.23 0.02
bottle 0.20 0.07
chair 0.01 0.26 0.24
tv 0.58 0.03
background 0.49 0.67 0.52 0.45 0.57 0.31 0.19 0.73 0.79 0.73 0.42
person car motorcycle bus truck dog cat bicycle bottle chair tv background
0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7

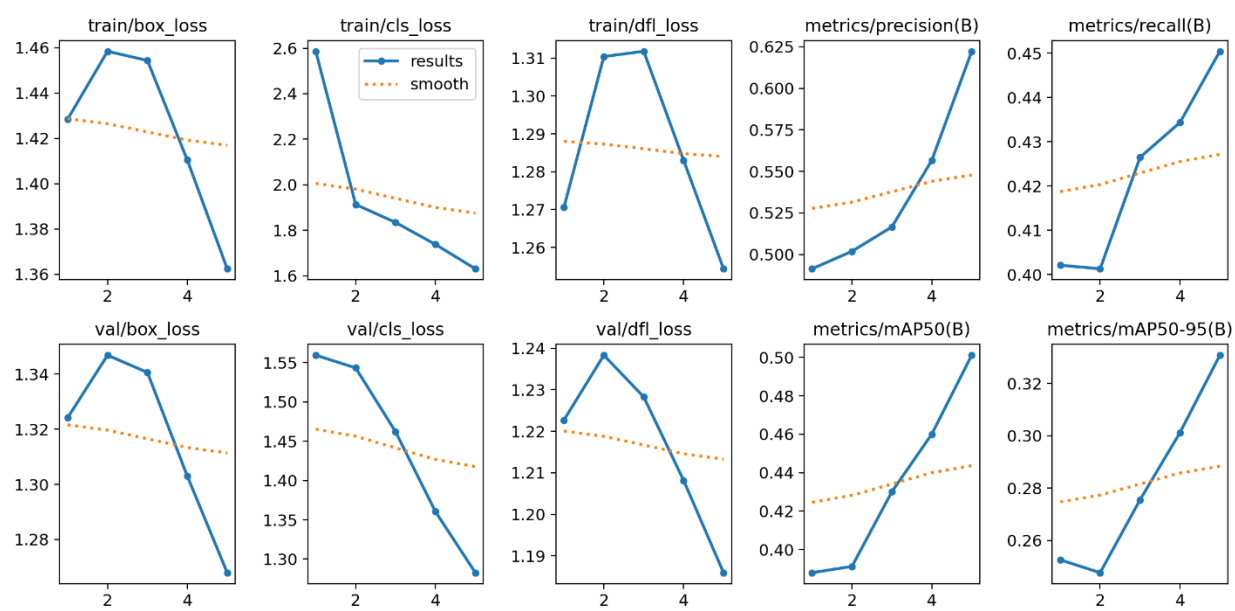
Task 3: Experiment - Resolution Comparison

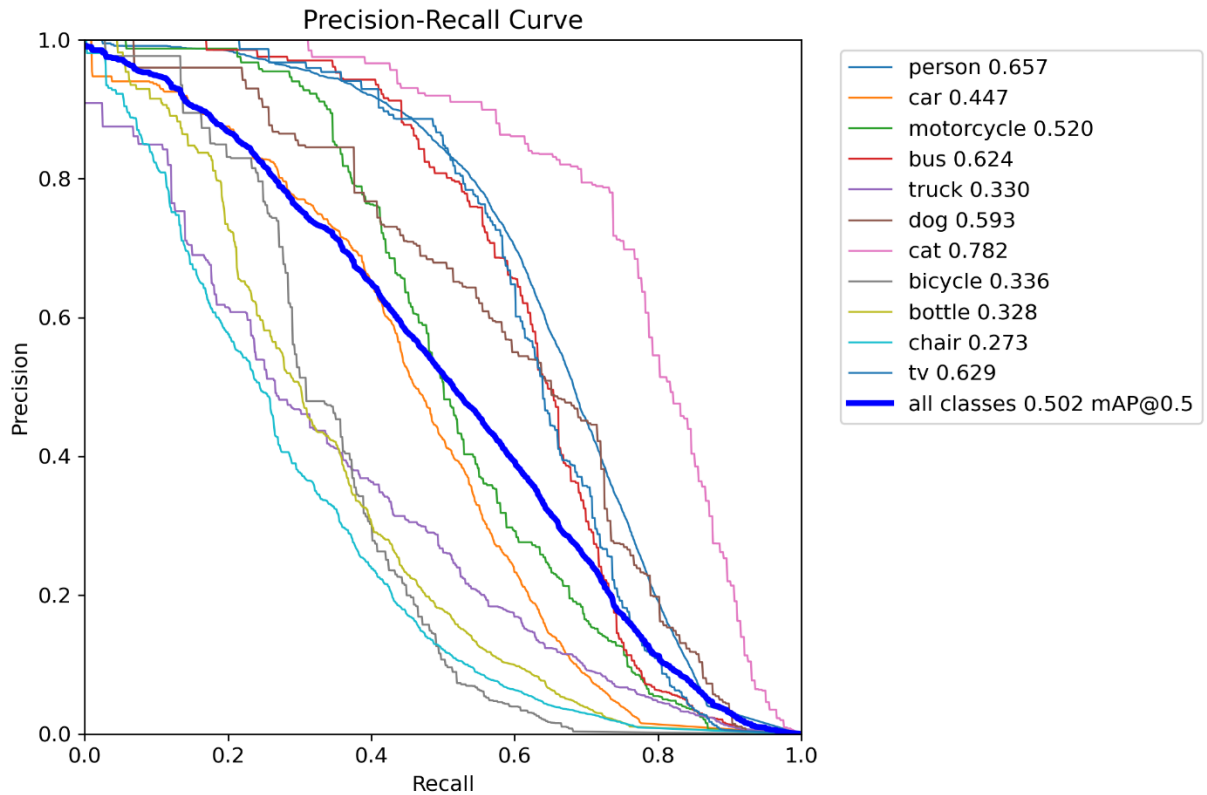
To investigate the effect of input resolution on detection performance, a controlled experiment was conducted by training two identical YOLOv8n models under the same conditions, differing only in input image size. The baseline was trained at imgsz=320 and the experiment model at imgsz=512. All other settings were kept constant: same 5 epochs, same dataset, same pretrained weights (yolov8n.pt), and the AdamW optimizer.

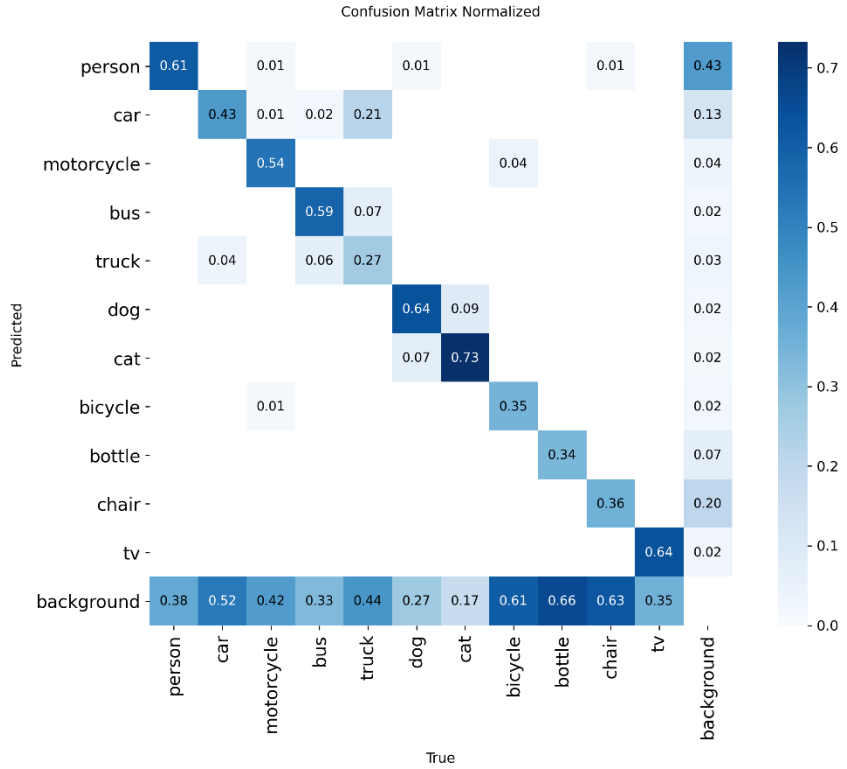
Higher resolution means the model sees more detail in each image, which generally improves localization accuracy for smaller or more complex objects. However it also increases memory usage and training time.

Metric	imgsz=320 (Baseline)	imgsz=512 (Experiment)	Winner
mAP@0.5	0.402	0.502	imgsz=512
mAP@0.5:0.95	0.2637	0.331	imgsz=512
Precision	0.5227	0.622	imgsz=512
Recall	0.4008	0.452	imgsz=512

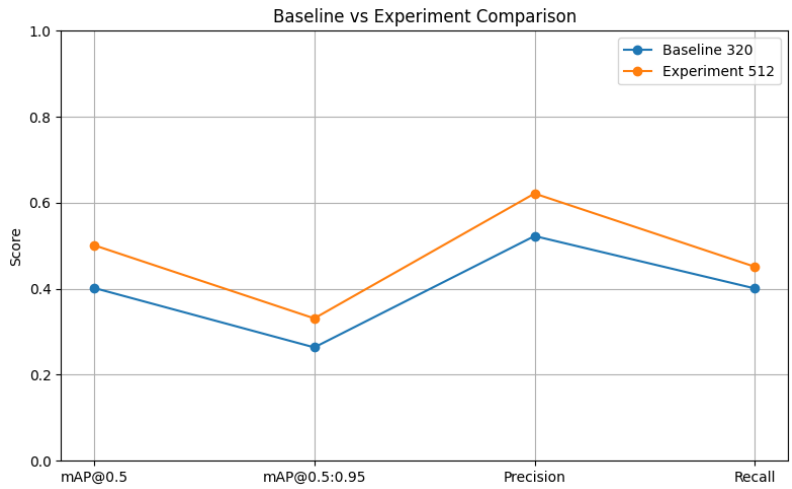
The imgsz=512 model outperformed the baseline across every metric. The most notable gain was in mAP@0.5, which improved by 25% (from 0.402 to 0.502). Recall also improved from 0.400 to 0.452, indicating the higher resolution model missed fewer objects. The main trade-off was higher training time, because the 512-model used larger feature maps and more computation per epoch.




Precision-Recall Curve
Precision
Recall
0.0
0.2
0.4
0.6
0.8
1.0
person 0.657
car 0.447
motorcycle 0.520
bus 0.624
truck 0.330
dog 0.593
cat 0.782
bicycle 0.336
bottle 0.328
chair 0.273
tv 0.629
all classes 0.502 mAP@0.5


Confusion Matrix Normalized
Predicted
True
person 0.61 0.01 0.01 0.01 0.43
car 0.43 0.01 0.02 0.21 0.13
motorcycle 0.54 0.04 0.04
bus 0.59 0.07 0.02
truck 0.04 0.06 0.27 0.03
dog 0.64 0.09 0.02
cat 0.07 0.73 0.02
bicycle 0.01 0.35 0.02
bottle 0.34 0.07
chair 0.36 0.20
tv 0.64 0.02
background 0.38 0.52 0.42 0.33 0.44 0.27 0.17 0.61 0.66 0.63 0.35
person car motorcycle bus truck dog cat bicycle bottle chair tv background
0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7

The comparison plot below summarizes the baseline and experiment results across the four main metrics.



Task 4: Evaluation

The best model weights from the imgsz=512 experiment run were used for final evaluation. Validation was performed on the full 3,532-image validation subset using the Ultralytics validation pipeline. For qualitative inference examples, detections were displayed at a confidence threshold of 0.25.

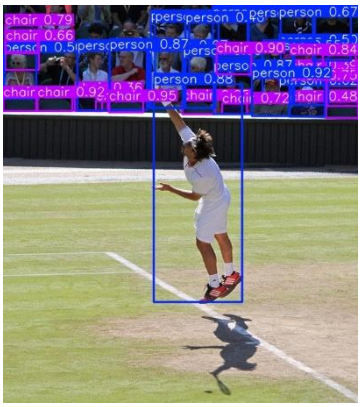
Overall Metrics

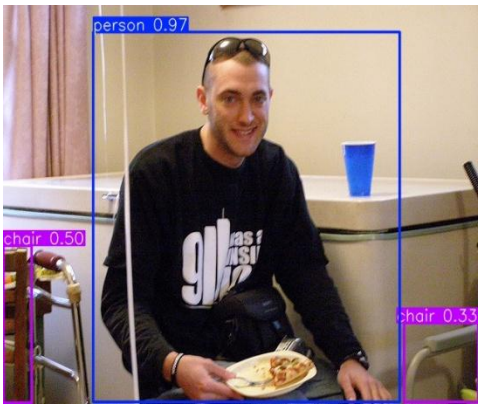
Metric	Value
mAP@0.5	0.502 (50.2%)
mAP@0.5:0.95	0.331 (33.1%)
Precision	0.622 (62.2%)
Recall	0.452 (45.2%)

The model achieved a mAP@0.5 of 50.2% on the validation set. The gap between mAP@0.5 and mAP@0.5:0.95 reflects the difficulty of precise localization — the model can detect objects correctly at a loose IoU threshold of 0.5, but bounding box accuracy drops under stricter thresholds. This is expected behavior for a model trained with only 5 epochs on a limited subset.

Task 5: Qualitative Analysis

Fifteen detection results were generated in total using the best imgsz=512 model. Three representative examples are shown below, and the full set of 15 detection images is included in the GitHub repository linked on page 1.





Analysis

The model performed well on clearly visible, large objects such as persons (confidence up to 0.97) and cats. Detections were accurate with tight bounding boxes for these classes. The motorcycle detection at confidence 0.27 shows the model was less certain — this is expected given motorcycles often appear partially occluded or at distance.

Some images returned no detections or only partial detections. This can happen when an image contains classes outside the selected 11, or when supported objects are small, occluded, or below the confidence threshold. This reflects the limits of a small model trained for only 5 epochs on a reduced dataset.

The main limitations observed are weaker performance on small objects like bottles and chairs, occasional false positives at low confidence thresholds, and imprecise box boundaries for overlapping objects. These limitations are directly tied to the short training duration (5 epochs) and the capped dataset size of 300 images per class. With more epochs and training data, detection quality would improve significantly.

Conclusion

This assignment demonstrated the complete pipeline for training YOLOv8n on a custom subset of MS COCO. Starting from annotation filtering and YOLO-format conversion, through training and evaluation, to qualitative detection analysis.

The baseline model (imgsz=320, 5 epochs) achieved a mAP@0.5 of 40.2%. The resolution experiment showed that increasing image size to 512 improved mAP@0.5 to 50.2%, a 25%

improvement with no other changes. This confirms that input resolution is a significant factor in detection accuracy, with the trade-off being higher computational cost.

Strong classes like cat, person, bus, and tv benefited most from higher resolution due to their distinct visual features. Smaller or visually ambiguous classes like bottle, chair and bicycle would require more training epochs and data to see meaningful improvement.